refactor_utils

Consolidate a Python package into a single, dependency-ordered file

Version 1.5.0

Generated 2026-11-09

Single-file PDF of the documentation site (docs/).

Contents

Home 3
Installation 6
Quickstart 7
CLI Reference 9
Modes 12
Configuration 14
Data Files 15
Examples 17
Project Files 19
Overview 25
Edge Cases 26
refactor_utils 29
cli 30
core 31
discovery 34
parser 36
resolver 37
writer 38
models 40
Contributing 42
Changelog 43

Home

Consolidate a small/medium Python module, library or project into one dependency-ordered file.

`pyrefactor` parses every file with **tree-sitter** (parallel workers), orders blocks by import-time dependencies, resolves collisions, merges imports, and emits a single `consolidated/<pkgname>.py` file — with full support for data files and source maps. (The older multi-file layouts remain available through the library API.)

```
pip install -e .
# Consolidate the package in the current directory (default)
pyrefactor
# Or point at a package directory
pyrefactor path/to/package
```

Output:

```
path/to/package/
  consolidated/
    <pkgname>.py   # imports -> constants -> classes -> functions -> entry points + __all__
```

The package name (`<pkgname>`) is detected from `pyproject.toml`/`setup.cfg`/`setup.py` in the package folder and up to two directory levels above it (covering `src/<pkgname>` layouts); when nothing is found, the current folder's name is used. `autoall` appends an `__all__` section to the generated file.

What it does

- **Parses** every file with tree-sitter in parallel; skips tests/`conftest.py`/`setup.py`/`__main__.py` (overridable via `--include`).
- **Orders** blocks by import-time deps (base classes, decorators, defaults, class bodies), preferring constants → classes → functions; cycles warned.
- **Resolves** same-named objects: shallowest keeps name, others become `<module>_<name>`; identical classes/functions optionally deduped (`--dedupe`).
- **Optimizes** imports: internal removed, external merged/de-duplicated/conflict-aliased, unused pruned (`--imports prune|keep|strict`), grouped future/stdlib/third-party.
- **Handles** `main()` and `if __name__ == "__main__":` guards (`--entry`, `--main-name`, `--generate-guard`).
- **Preserves** data files (`.json`, `.csv`, `.ini`, etc.) via `--copy-data`.
- **Logs** with loguru (`--log-level`, `--log-file`).

Quick Links

- Installation
- Quickstart
- CLI Reference
- Project Files — every source file documented
- Architecture
- Examples
- API Reference

Features at a Glance

Capability	Flag	Default
Dry-run / check	--dry-run / --check	off
Stdout	--stdout	off
Config	--config pyproject.toml / --pyproject	auto
Formatting	--format / --formatter black|ruff	off
Diff	--diff	off
Overwrite	--overwrite / --force	warn
Include vs exclude	--include GLOB / --exclude GLOB	precedence: default < include < exclude
Data files	--copy-data / --data-glob	off
Docstrings/comments	--strip-docstrings / --strip-comments	keep
Collisions	--rename-strategy prefix|sufix|module_path|error	prefix
Dedupe	--dedupe none|safe|aggressive	safe
Imports	--imports prune|keep|strict	prune
__all__	--all ignore|respect|merge|recompute	ignore
Entry	--entry / --main-name / --generate-guard	__
Namespaces	--namespace / --preserve-modules / --source-map	off
Strictness	--fail-on-warning / --max-skipped / --allow-syntax-errors	off
Parallelism	--workers / --jobs / --no-parallel / --progress	cpu count

Capability	Flag	Default
Validation	--validate / --validate-import	off

License

MIT — see GitHub.

Installation

Requirements

- Python ≥ 3.10
- `tree-sitter` + `tree_sitter_python`
- `loguru`

From Source

```
git clone https://github.com/i4147/refactor_utils.git
cd refactor_utils
pip install -e .
```

Extras

```
pip install -e ".[format]"   # black + ruff for --format
pip install -e ".[progress]"  # tqdm for --progress
pip install -e ".[test]"      # pytest + coverage
pip install -e ".[docs]"      # mkdocs + material + mkdocstrings
```

Or all:

```
pip install loguru tree-sitter tree_sitter_python black ruff tqdm pytest mkdocs mkdocs-material
"mkdocstrings[python]"
```

Verify

```
pyrefactor --help
pyrefactor --version
pytest -q
mkdocs serve
```

Development Setup

```
pip install -e ".[test,docs,format,progress]"
pre-commit install  # if configured
```

Troubleshooting

- **`externally-managed-environment`** on Debian/Ubuntu: `pip install -e . --break-system-packages` or use `pipx`/venv.
- **`ModuleNotFoundError: tree_sitter`**: ensure `tree-sitter` and `tree_sitter_python` are installed for your Python.
- **Black/ruff not found for `--format`**: install via `pip install black ruff`; otherwise formatting is skipped with a warning.

Quickstart

Basic consolidation

```
# 1. Run inside the package folder — the path defaults to "."
cd src/mypackage
pyrefactor
# -> src/mypackage/consolidated/mypackage.py

# 2. …or point at the package from anywhere
pyrefactor src/mypackage
# -> src/mypackage/consolidated/mypackage.py

# 3. …or choose another output directory
pyrefactor src/mypackage -o dist
# -> dist/mypackage.py
```

There is no `-m`/`--mode` flag any more: one call always produces one file, `<pkgname>.py`, inside a `consolidated/` subdirectory.

The generated file contains, in this order:

1. a module docstring,
2. `__future__` imports and the optimized import section (de-duplicated, relative/local imports stripped),
3. constants (`const.py`),
4. classes (`classes.py`),
5. functions (`func.py`),
6. entry points and `if __name__ == "__main__":` guards,
7. an `__all__` section generated by `autoall` in place.

The package name used for the file name is detected from `pyproject.toml`/`setup.cfg`/`setup.py` in the package folder **and up to two directory levels above it** (so `src/<pkgname>` layouts work). When nothing is found, the name of the folder you pointed at is used. Names are normalised to valid module names: `my-dist` → `my_dist`.

Dry-run & check (CI)

```
pyrefactor src/mypackage --dry-run          # report, write nothing
pyrefactor src/mypackage --check --diff     # exit 2 when the file would change
```

`--check` compares the generated file byte-for-byte (including the `__all__` section) and prints a unified diff when it differs. It is idempotent: run it twice after a successful build and it passes.

Stdout & diff

```
pyrefactor src/mypackage --stdout > consolidated.py
pyrefactor src/mypackage --diff            # print the diff against the file
```

Data files

```
pyrefactor src/mypackage --copy-data
pyrefactor src/mypackage --copy-data --data-glob "*.json"
```

Data files (`.json`, `.yaml`, `.csv`, `.txt`, `.ini`, `.toml`, …) are discovered via `discovery.discover_data_files()` and copied next to the generated module, preserving their path **relative to the package directory** so `Path(__file__).parent / "data/…"` keeps working.

Formatting & validation

```
pyrefactor src/mypackage --format --formatter black --validate
pyrefactor src/mypackage --validate-import
```

Include / exclude

Precedence: `default excludes < --include < --exclude`

```
pyrefactor src/mypackage --exclude "tests/*" --include "conftest.py"
```

Entry points

```
pyrefactor src/mypackage --entry mypackage.cli
pyrefactor src/mypackage --entry-keep-all
pyrefactor src/mypackage --main-name cli_main --generate-guard
```

Namespaces & source maps (debugging)

```
pyrefactor src/mypackage --source-map --preserve-modules --namespace
```

Adds per-block:

```
# Original module: pkg.sub.mod
# source: src/pkg/sub/mod.py:10-25
# namespace: pkg.sub.mod
def foo(): ...
```

Configuration file

`pyproject.toml` next to (or above) the package is read automatically:

```
[tool.pyrefactor]
exclude = ["tests/*", "ignored.py"]
entry = ["mypackage.cli"]
rename_strategy = "suffix"
dedupe = "safe"
imports = "prune"
all = "respect"
format = true
```

```
pyrefactor src/mypackage                      # auto-detected
pyrefactor src/mypackage --config other.toml  # explicit file
pyrefactor src/mypackage --main-name run      # CLI flag wins over config
```

Python API

```
from pathlib import Path
from refactor_utils.core import consolidate_package

# Single consolidated file: src/mypackage/consolidated/mypackage.py
result = consolidate_package(Path("src/mypackage"), validate=True)
print(result.modules, result.exports, result.pkg_name, result.files)

# Custom output directory: dist/mypackage.py
result = consolidate_package(Path("src/mypackage"), Path("dist"))

# Everything the CLI does is available as a keyword argument
result = consolidate_package(
    Path("src/mypackage"),
    rename_strategy="suffix",
    entry=("cli",),
    main_name="run",
    copy_data=True,
    do_format=True,
    formatter="black",
    check=False,
)
```

The lower-level `refactor_utils.core.consolidate()` (multi-file `output/` layouts driven by `mode="single"|"package"|"module"|"flat"|"split"`) is still available for library users; the `pyrefactor` CLI always produces the single-file consolidated layout.

CLI Reference

Generated from `pyrefactor --help`. Every flag lists the module it touches.

```
usage: pyrefactor [-h] [-o OUTPUT] [-j WORKERS] [--jobs JOBS]
                  [--executor {process,thread}] [--no-parallel] [--progress]
                  [--progress-bar] [-x GLOB] [--include GLOB] [--copy-data]
                  [--no-copy-data] [--data-glob GLOB] [--entry MODULE]
                  [--entry-keep-all] [--main-name NAME] [--generate-guard]
                  [--keep-all-mains] [--keep-unused-imports]
                  [--imports {prune,keep,strict}] [--respect-all]
                  [--all {ignore,respect,merge,recompute}]
                  [--rename-strategy {prefix,suffix,module_path,error}]
                  [--dedupe {none,safe,aggressive}] [--keep-docstrings]
                  [--strip-docstrings] [--keep-comments] [--strip-comments]
                  [--namespace] [--preserve-modules] [--source-map]
                  [--dry-run] [--check] [--stdout] [--diff] [--overwrite]
                  [--force] [--config PATH] [--pyproject] [--setup-cfg]
                  [--format] [--post-format] [--formatter {black,ruff}]
                  [--validate] [--validate-import] [--strict]
                  [--fail-on-warning] [--max-skipped N]
                  [--allow-syntax-errors] [--eol {lf,crlf,preserve}]
                  [--log-level {TRACE,DEBUG,INFO,WARNING,ERROR}]
                  [--log-file LOG_FILE] [--version]
                  [source]
```

Positional

- `source` — package directory (or single `.py` file) to consolidate; defaults to the current directory.

Output

- `-o OUTPUT, --output OUTPUT` — output directory (default: `<pkg>/consolidated`). A single `<pkgname>.py` file is written inside it.

The CLI no longer accepts `--mode`; it always produces the consolidated single-file layout (`consolidated/<pkgname>.py`) described in Quickstart. The legacy multi-file layouts (`mode="package"|"module"|"flat"|"split"`) remain available through the library API (`refactor_utils.core.consolidate`).

`[tool.pyrefactor]` in `pyproject.toml` is read automatically (or from `--config PATH`); values only fill in flags that were not passed explicitly.

Exit codes

Code	Meaning
0	success (or --check found no drift)
1	unexpected failure, or strictness rule triggered (--strict, --fail-on-warning, --max-skipped)
2	bad input / build problem (ConsolidationError, including a failing --check)

Parallelism

- `-j WORKERS, --workers WORKERS` — workers (default CPU).

- `--jobs JOBS` — alias for `--workers` (`cli.py`).
- `--executor {process,thread}` — executor type.
- `--no-parallel` — disable parallel (`workers=1`).
- `--progress` / `--progress-bar` — tqdm bar (falls back to log).

Discovery

- `-x GLOB, --exclude GLOB` — skip glob (repeatable) — `discovery.py`.
- `--include GLOB` — re-include; precedence `default < --include < --exclude`.
- `--copy-data` / `--no-copy-data` — copy data files preserving paths relative to the package (opt-in; off by default for the `consolidated/` layout).
- `--data-glob GLOB` — only copy data matching glob (repeatable, matches file name or relative path).

Entry

- `--entry MODULE` — keep only guards of listed modules (repeatable) — `resolver.py`.
- `--entry-keep-all` / `--keep-all-mains` — keep all guards.
- `--main-name NAME` — rename `main()` → `NAME`.
- `--generate-guard` — auto `if __name__ == "__main__": main()`.

Imports / Exports

- `--keep-unused-imports` — deprecated alias for `--imports keep`.
- `--imports {prune,keep,strict}` — `prune` drop unused (default), `keep` retain, `strict` warn.
- `--respect-all` — deprecated alias for `--all respect`.
- `--all {ignore,respect,merge,recompute}` — `__all__` policy.

Collisions

- `--rename-strategy {prefix,suffix,module_path,error}` — `prefix` → `<mod>_name` (default).
- `--dedupe {none,safe,aggressive}` — duplicate removal.

Comments / Namespaces

- `--keep-docstrings` (default) / `--strip-docstrings` — via AST — `parser.py`.
- `--keep-comments` (default) / `--strip-comments` — keep shebang/encoding even when stripping.
- `--namespace` — `# namespace: module` per block — `writer.py`.
- `--preserve-modules` — `# Original module: ...`.
- `--source-map` — `# source: path:line-line`.

Output Control

- `--dry-run` — plan only, never write — `core.py`.
- `--check` — like dry-run but fail if diff non-empty.
- `--stdout` — to stdout not files.
- `--diff` — unified diff vs existing.
- `--check` — fail when the output would change **or does not exist yet** (like `black --check` / `isort --check`).
- `--overwrite` / `--force` — allow overwrite non-empty.
- `--eol {lf,crlf,preserve}` — line endings for generated files; `preserve` matches the dominant ending of the input files — `core.py`.

Config / Format / Validate

- `--config PATH` — `pyproject.toml` with `[tool.pyrefactor]` — `cli.py`/`core.py`/`discovery.py`.
- `--pyproject` / `--setup-cfg` — auto-detect near source.
- `--format` / `--post-format` — `black`/`ruff` — `writer.py`.
- `--formatter {black,ruff}`.
- `--validate` — compile; `--validate-import` also imports the generated module. The import **executes top-level code** and logs a prominent warning first — `core.py`.

Strictness

- `--strict` — exit 1 if any file was skipped, and refuse to write output when a warning means the result may be wrong (unresolved name lookups, computed `__all__`, unresolvable internal imports) or when an import cycle has to be broken.
- `--fail-on-warning` — exit 2 if warnings.
- `--max-skipped N` — fail if `> N` skipped.
- `--allow-syntax-errors` — partial parse — `parser.py`.
- `--eol` — `lf` (default), `crlf`, or `preserve` — `core.py`.

Logging

- `--log-level {TRACE,DEBUG,INFO,WARNING,ERROR}` (default `INFO`).
- `--log-file LOG_FILE`.

Examples

```
pyrefactor src/pkg -o out --dry-run --log-level INFO
pyrefactor src/pkg -o out --include "conftest.py" --exclude "tests/*"
pyrefactor src/pkg -o out --rename-strategy suffix --dedupe aggressive --all merge
pyrefactor src/pkg -o out --source-map --preserve-modules --validate
pyrefactor src/pkg -o out --mode split --copy-data --format
```

Modes

CLI layout: `consolidated/<pkgname>.py` (only layout of `pyrefactor`)

```
pyrefactor src/mypackage
```

```
src/mypackage/
  consolidated/
    mypackage.py     # the single generated module
```

`mypackage.py` contains (in order) the module docstring, `__future__` and optimized imports, constants, classes, functions, entry points/guards and the `autoall`-generated `__all__` section. The output directory always holds exactly one file: temporary `const.py`/`classes.py`/`func.py` extraction files are removed after the merge, and files from the legacy `output/` layout are cleaned up when the directory is called `consolidated/` or `output/`.

`core.consolidate_package()` implements this layout; `writer.render_consolidated_single()` renders the body and `autoall.ensure_all_file()` appends `__all__` in place.

Library layouts: `core.consolidate(mode=...)`

The older, lower-level API still supports five layouts. They are not reachable from the CLI, which keeps `pyrefactor` to a single, predictable output.

`single` (default of `consolidate()`)

```
output/
  __init__.py        # from ._consolidated import ... ; __all__ = [...]
  _consolidated.py   # docstring, future, imports, guards, constants, classes, functions, entry points
```

`core.py` calls `writer.render_consolidated()` + `render_init()`.

`package`

```
output/<pkg>/
  __init__.py
  _consolidated.py
  (data preserved under output/<pkg>/...)
```

Uses `render_package()`; package name derived from source dirname/stem.

`module`

Single file, no `__init__.py`:

```
output/_consolidated.py  # or output (if path ends with .py, that file)
```

`render_module()`; `--stdout` gives the same text on stdout.

`flat`

No section headers, minimal spacing:

```
"""Flat consolidated from ``pkg``."""
from __future__ ...
import os
X = 1
class Bar: ...
```

`render_flat()`; still includes `__init__.py` for compatibility.

`split`

By kind:

```
output/
  const.py   # constants + future + imports
  class.py   # classes
  func.py    # functions
  main.py    # entry points
  __init__.py
```

`render_split()` returns a dict `fname → content`.

Choosing a layout

- **CLI / publishing a module**: the default `consolidated/<pkgname>.py`.
- **Library users that need the old multi-file tree**: `consolidate(mode=...)` with `single`, `package`, `module`, `flat` or `split`.

All layouts compile (`compile(..., "exec")`) and copy data files under the output root (or package dir).

Configuration

Persist settings in `pyproject.toml` under `[tool.pyrefactor]`.

Example `pyproject.toml`

```
[tool.pyrefactor]
exclude = ["tests/*", "docs/*", "ignored.py"]
include = ["tests/fixtures/*.py"]
entry = ["mypackage.cli"]
rename_strategy = "suffix"      # prefix|sufix|module_path|error
dedupe = "safe"                 # none|safe|aggressive
imports = "prune"               # prune|keep|strict
all = "respect"                 # ignore|respect|merge|recompute
format = false
fail_on_warning = false
max_skipped = 5
allow_syntax_errors = false
```

Loading

```
pyrefactor src/pkg                          # auto-detects the nearest pyproject.toml
pyrefactor src/pkg --config other.toml      # explicit file
pyrefactor src/pkg --main-name run          # CLI flag wins over config

cli.apply_config():
```

1. If `--config PATH` is given, load that file's `[tool.pyrefactor]` (`cli._read_tool_config()`).
2. Otherwise search upwards from the source path with `discovery.detect_pyproject_config()`.
3. Merge: a flag whose value differs from the argparse default is treated as explicit and wins; config values fill in everything else.

Supported keys: `exclude`, `include`, `entry`, `entry_keep_all`, `main_name`, `generate_guard`, `rename_strategy`, `dedupe`, `imports`, `all`, `format`, `fail_on_warning`, `max_skipped`, `copy_data`, `data_glob`, `namespace`, `preserve_modules`, `source_map`, `keep_docstrings`, `strip_comments`, `keep_comments`, `eol`, `strict`, `workers`.

Example for `refactor_utils` Itself

See repo root `pyproject.toml`:

```
[tool.pyrefactor]
exclude = ["tests/*", "test_*.py", "iwgfyrkugf.py"]
rename_strategy = "prefix"
dedupe = "safe"
imports = "prune"
all = "ignore"
max_skipped = 10
```

See Also

- `discovery.detect_pyproject_config()` — loads TOML (tomllib/tomli).
- `cli.apply_config()` — merging logic (CLI wins over config).

Data Files

`pyrefactor` preserves non-Python assets alongside code — JSON configs, CSV datasets, templates, etc.

Discovery

`discovery.discover_data_files(root, exclude, include, skip_dirs, copy_data)`:

- Walks source tree (respecting `--include`/`--exclude` precedence `default < include < exclude`).
- Identifies data via `DATA_EXTENSIONS` and `DATA_FILENAMES`:

```
DATA_EXTENSIONS = {
    ".json",
    ".yaml",
    ".yml",
    ".toml",
    ".ini",
    ".cfg",
    ".txt",
    ".csv",
    ".tsv",
    ".xml",
    ".html",
    ".css",
    ".js",
    ".md",
    ".pickle",
    ".db",
    ...,
}
DATA_FILENAMES = {
    "requirements.txt",
    "Pipfile",
    "setup.cfg",
    "pyproject.toml",
    "LICENSE",
    ...,
}
```

- Returns `list[DataFile(path, relative)]` where `relative` is `path.relative_to(base)` (base = `detect_base(root)` → project root or `src/`).

If the project had no data files, returns `[]` (no-op).

Copy Semantics

`core.consolidate(..., copy_data=True, data_glob=()):`

```
for df in data_files:
    relative = df.path.relative_to(source_dir)   # package-relative
    dest = out_dir / relative                    # consolidated/data/config.json
    dest.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(df.path, dest)                  # preserves mtime
```

`consolidate_package()` always uses paths relative to the **package** directory, so a module that does `Path(__file__).parent / "data" / "config.json"` keeps working after consolidation (`__file__` is now `consolidated/<pkgname>.py`).

Filtered by `--data-glob` (repeatable) via `fnmatch`:

```
pyrefactor src/app -o output --data-glob "*.json" --data-glob "data/*.csv"
```

With `--no-copy-data`, discovery returns `[]` and nothing is copied.

Example

```
source/
  pkg/
    __init__.py
    core.py
    data/
      config.json
    settings.ini

pkg/
  consolidated/
```

```
    pkg.py
    data/config.json      # package-relative path preserved
    settings.ini
```

From repo's own `src/refactor_utils/data/`:

```
pyrefactor src/refactor_utils -o /tmp/out --copy-data --validate
ls /tmp/out/data/
# README.md  config.ini  example.csv  sample.json
```

Examples & Tests

- `examples/data_files_example/` — runnable demo with `mypackage/data/config.json`.
- `examples/config_example/` — TOML filtering.
- `tests/test_data_files.py` — 6 cases (copy, no-copy, glob, structure, modes, exclude).
- `tests/test_discovery.py::test_discover_data_files*` — unit.

When to Disable

- Binary assets already managed externally.
- Output is a single file (`--mode module`) where data not needed (use `--no-copy-data`).

See Also

- `discovery.py` — `DATA_EXTENSIONS`, `discover_data_files()`
- `core.py` — copy loop + `copy_data`/`data_glob` params
- `src/refactor_utils/data/README.md` — included example data

Examples

Runnable examples in `examples/` (see `examples/README.md`).

Basic Usage (`examples/basic_usage.py`)

CLI:

```
pyrefactor examples/data_files_example/mypackage -o /tmp/basic_out --overwrite --validate
```

Python API:

```
from pathlib import Path
from refactor_utils import consolidate

result = consolidate(Path("mypackage"), Path("output"))
print(result.modules, result.exports)
```

Run:

```
python examples/basic_usage.py
```

Advanced Flags (`examples/advanced_flags.py`)

```
consolidate(pkg, out, rename_strategy="suffix")
consolidate(pkg, out, dedupe="aggressive")
consolidate(pkg, out, all_mode="merge")
consolidate(pkg, out, source_map=True, preserve_modules=True, namespace=True)
```

Run:

```
python examples/advanced_flags.py
```

Programmatic API (`examples/programmatic_api.py`)

- `dry_run=True` — plan without writes.
- `validate=True` + `validate_import=True`.
- `diff=True` vs existing.
- `data_glob=("*.json",)`.

```
python examples/programmatic_api.py
```

Data Files Example (`examples/data_files_example/`)

```
mypackage/
  __init__.py
  core.py      # uses Path(__file__).parent / "data" / "config.json"
  utils.py
  data/
    config.json
    sample.csv
```

```
pyrefactor examples/data_files_example/mypackage -o /tmp/out --copy-data
ls /tmp/out/mypackage/data/config.json
pyrefactor examples/data_files_example/mypackage -o /tmp/out --data-glob "*.json"
```

Config Example (`examples/config_example/`)

`pyproject.toml`:

```
[tool.pyrefactor]
exclude = ["tests/*", "ignored.py"]
rename_strategy = "suffix"
```

```
pyrefactor ./src -o output --config examples/config_example/pyproject.toml
pyrefactor ./src -o output --pyproject  # auto-detect
```

Integration Tests as Examples

`tests/test_integration.py` shows:

- `src/` layout, namespace packages, circular imports, large projects (20 files), output-inside-source, programmatic API.

Run:

```
pytest tests/test_integration.py -v
```

Project Files

Every file in `src/refactor_utils/` and top-level.

`pyproject.toml`

- `[build-system]` setuptools.
- `[project]` name `refactor_utils`, version `1.5.0`, deps `loguru`, `tree-sitter`, `tree_sitter_python`; optional `[format]`, `[progress]`, `[test]`, `[docs]`, entry point `pyrefactor = refactor_utils.cli:main`.
- `[tool.setuptools]` where=`src`, package-data `data/*`.
- `[tool.pyrefactor]` default config for this repo (exclude `tests/*`, `iwgfyrkugf.py`).
- `[tool.pytest] testpaths = ["tests"]`, `pythonpath = ["src"]`.
- `[tool.black]`/`[tool.ruff]`/`[tool.coverage]`/`[tool.mkdocstrings]`.

`src/refactor_utils/__init__.py`

Public API: re-exports `ConsolidationError`, `ConsolidationResult`, `consolidate` and `consolidate_package` from `core.py`, plus `__version__` (1.5.0).

`src/refactor_utils/cli.py`

`build_parser()` defines all flags with help referencing the module that implements them. `apply_config(args, parser)` merges `[tool.pyrefactor]` (explicit `--config` or auto-detected via `discovery.detect_pyproject_config()`); a flag that differs from its argparse default is treated as explicit and wins. `main()` resolves aliases (`--jobs`→`--workers`, `--progress-bar`, `--keep-all-mains`, `--post-format`), calls `consolidate_package()`, and maps failures to exit codes (`ConsolidationError` → 2, unexpected → 1). `_read_tool_config(path)` parses an explicit TOML file.

`src/refactor_utils/core.py`

Orchestrates `discover → _parse_all (parallel) → Resolver → writer`.

- `_parse_worker` (picklable), `_parse_all` (Process/Thread pool with BrokenExecutor fallback, tqdm).
- `ConsolidationResult` (fields: `output_dir`, `files`, `modules`, `skipped`, `exports`, `warnings`, `diff`, `stdout_text`, `copied_data_files`, `validated`, `dry_run`, `plan`, `all_added`, `pkg_name`).
- `consolidate_package(source=".", output=None, **options)` — the `pyrefactor [pkg]` contract (see Quickstart): package-name detection, `consolidated/<pkgname>.py` output, temporary `const.py`/`classes.py`/`func.py` extraction (`_write_section_files`, `_merge_body`), `__all__` added in place by `autoall` (`_render_final`), stale-output cleanup (`_cleanup_stale_outputs`), `--diff` (`_unified_diff`) and optional formatting (`writer.format_source`).
- `consolidate(...)` — the lower-level multi-layout API (`mode="single"|"package"|"module"|"flat"|"split"`) kept for library users.

`src/refactor_utils/discovery.py`

- `EXCLUDED_DIRS` (`.git`, `__pycache__`, `venv`, `build`, `tests`, `docs`, ...)
- `EXCLUDED_FILE_GLOBS` (`__main__.py`, `conftest.py`, `test_*.py`, `*_test.py`, `setup.py`, `iwgfyrkugf.py`, `_consolidated.py`)
- `DATA_EXTENSIONS`/`DATA_FILENAMES` (json/yaml/csv/txt/.../LICENSE)

- `detect_base()` — handles `__init__.py` parent and `src/` layout.
- `_is_excluded()` — precedence `default < include < exclude`.
- `_walk()` — iterative, symlink-aware, debug logs.
- `discover()` — returns `list[SourceFile(path, module, is_pkg)]` with `detect_base` and `relative_to(base)`, handles single-file, namespace (no `__init__.py`), output-inside-source skip.
- `discover_data_files()` — non-py data discovery, respects same filters, returns `DataFile(path, relative)`.
- `detect_pyproject_config()` — finds `pyproject.toml` upwards, loads `tomllib`/`tomli` `[tool.pyrefactor]`.
- `normalize_pkg_name(name)` — turns a distribution/folder name into a valid module name (`my-dist` → `my_dist`, leading digit prefixed with _).
- `detect_pkg_name(root)` — reads `pyproject.toml`/`setup.cfg`/`setup.py` in `root` and up to two levels above (covering `src/<pkgname>`), falling back to the folder name; always returns a normalised identifier.
- `discover_data_files(..., data_glob=())` — optional glob filter matched against the file name and its relative path.

src/refactor_utils/parser.py

Tree-sitter based `parse_module_file(path_str, module, is_pkg, keep_docstrings, strip_comments, keep_comments, allow_syntax_errors)` → `ModuleInfo`.

- `_get_parser()` thread-local `Parser(Language(tree_sitter_python))`.
- `parse_import()`, `_resolve_relative()`, `_dotted()`, `_binding_names()`, `_param_names()`, `_lhs_names()`, `_target_names()`, `_bound_names()`, `_scan()`, `_extent()`, `_first_error_line()`, `_collect_occs()` (identifier/attribute, import nested, function/lambda/class collection with shadow/lazy, class body block handling), `_detect_dynamic_patterns`/`_detect_runtime_semantics` (importlib, eval, **file**, etc.), `_strip_docstring_from_source` (AST), `_strip_comments_from_source` (shebang-preserving), `_extract()` (orchestrates blocks `CONST|CLASS|FUNC|MISC|MAIN`, guards, `__all__`, TYPE_CHECKING, conditional warnings).

Handles BOM, CRLF warning, non-UTF8 decode, `allow_syntax_errors` partial parse.

src/refactor_utils/models.py

Picklable dataclasses for worker communication:

- `CONST`, `CLASS`, `FUNC`, `MISC`, `MAIN`
- `ImportSpec(module, name, alias, is_from, star, relative)` + `local_name`
- `Occ(start, end, name, kind, eager, obj, specs)` (attr chain)
- `Block(module, kind, names, source, occs, order, bid, orig_path, start_line, end_line, stripped_docstring)` + `merge()`
- `GuardBlock(text, specs)`
- `ModuleInfo(module, path, is_pkg, imports, blocks, guards, declared_all, future_annotations, warnings, errors)`
- `SourceFile(path, module, is_pkg)`

- `DataFile(path, relative, kind)`
- `ConsolidationOptions(keep_docstrings, strip_comments, ..., rename_strategy, ...)` — bundles options for workers.

src/refactor_utils/resolver.py

Cross-module `Resolver(modules, keep_unused_imports, respect_all, entry, workers, rename_strategy, dedupe, imports_mode, all_mode, namespace, preserve_modules, source_map, entry_keep_all, main_name, generate_guard)`.

- `_prefix`/`_suffix`/`_module_path_prefix`/`_sanitize`.
- `_spec_target`, `_resolve_symbol`, `_resolve_dotted` — internal vs external, star expansion.
- `build()` → `_build_envs`, `_analyse` (eager/proj/ext_keys), `_dedupe` (safe/aggressive/none), `_emit_keys` via `imports_mode`, `_assign_names` (collision → rename_strategy/error, `main_name` override), `_resolve_ext_conflicts` (import alias), `_dedupe_post_rename` (`_block_signature`, fixed point), `_render_all`, `_assemble` (topo_order, tail mains, generate_guard, guards), `_exports` (`all_mode`).
- `Plan(future_names, imports_text, guards, blocks, exports, renamed, deduped, warnings)`, `RenderedBlock(section, kind, text, origin, module)`.

src/refactor_utils/ordering.py

`topo_order(nodes, deps, rank, key)` — heap-based topological sort grouping by `rank` (CONST/MISC < CLASS < FUNC), cycle detection with warning `Circular import-time dependency: emitting X before Y`.

src/refactor_utils/writer.py

- `_sep` (spacing), `_format_with_black`/`_format_with_ruff`, `format_source(text, formatter)` (public wrapper used by `consolidate_package(--format)`).
- `render_consolidated(plan, source_name, namespace, preserve_modules, source_map, do_format, formatter)` — section headers `Constants/Classes/Functions/Entry points`, per-block prefix comments (`# Original module:`, `# source:`, `# namespace:`), formatting.
- `render_init(plan, source_name, module)` — `from ._consolidated import (...)` + `__all__`.
- `render_flat`, `render_split` (dict `const.py`/`class.py`/etc.), `render_package`, `render_module`.

src/refactor_utils/imports.py

- `render_import_line(ImportSpec)`, `_group` (stdlib vs third-party via `sys.stdlib_module_names`), `_item_key` (constant/upper/lower sorting), `_wrap` (88-col), `future_section`, `build_import_section(keys, ext_final)` — groups `plain` (import) and `froms`, handles `star`, sorting, wrapping.

src/refactor_utils/logging_setup.py

`setup_logging(level, log_file)` — `loguru` remove/add stderr with green/time format, optional file sink `DEBUG` 5MB rotate.

src/refactor_utils/ordering.py

(see above) + uses `loguru`.

src/refactor_utils/data/

Example data files demonstrating preservation:

- `sample.json` — `{"project": "refactor_utils", ...}`

- `example.csv` — `id,name,value`
- `config.ini` — `[pyrefactor]` example
- `README.md` — explains handling (see `docs/data-files.md`).

`src/refactor_utils.egg-info/` & `src/refactor_utils/iwgfyrkugf.py`

- Egg-info generated by `setuptools` (PKG-INFO, SOURCES.txt, entry_points).
- `iwgfyrkugf.py` — vendored previous consolidation example, excluded via `EXCLUDED_FILE_GLOBS` to avoid self-loop.

Top-Level

- `prompt.md` — original AI agent task (features/edge cases to implement).
- `mkdocs.yml` — Material theme, nav, plugins (`search`, `mkdocstrings`).
- `docs/` — see Architecture; `docs/README.md` explains how to build the HTML docs and the single-file PDF.
- `tests/` — pytest suite (discovery, parser, resolver, writer, cli, core, data, integration). `tests/test_consolidated.py` covers the `consolidated/<pkgname>.py` flow, `tests/test_cli_workflow.py` drives the CLI end to end (no-arg run, `--check`, config merge, exit codes) and `tests/test_autoall.py` pins the in-place `__all__` behaviour.
- `examples/` — runnable demos (basic, advanced, programmatic, data_files_example, config_example).
- `arena/` — helper scripts (bash/python) used during this session; see `arena/README.md`. Includes the docstring audit, the PDF builder and the repo-wide consolidation demos.
- `refactor_utils.pdf` — generated single-file documentation PDF (see `docs/README.md`).
- `site/` — generated HTML documentation (git-ignored).
- `.gitignore` — ignores `output/`, `output_*/`, `.venv/`, `__pycache__/`, etc.

`src/refactor_utils/discovery.py` (additions)

- `detect_pkg_name(root)` — detects the consolidated package name by reading `pyproject.toml` (`[project] name`), `setup.cfg` (`[metadata] name`) or `setup.py` (`setup(name=...)`) in the package folder and up to two directory levels above it (covering `src/<pkgname>` layouts); falls back to the folder's own name.

`src/refactor_utils/core.py` (additions)

- `consolidate_package(source, output=None, **kwargs)` — implements the `pyrefactor [pkg]` contract: detects the package name, creates a `consolidated` output directory, runs the existing `discover → parse → resolve` pipeline, writes

temporary `const.py` / `classes.py` / `func.py` section files, assembles them (plus the optimized, local-imports-stripped import section and entry points) into a single `consolidated/<pkgname>.py`, removes the temporaries, and delegates `__all__` generation to `autoall.ensure_all_file`. Returns a `ConsolidationResult` extended with `all_added` and `pkg_name`.

`src/refactor_utils/writer.py` (additions)

- `render_consolidated_single(plan, source_name)` — renders one module file: `__future__` → optimized imports → `TYPE_CHECKING` guard → constants → classes → functions → entry points, with per-block namespace/source-map comments when requested.

`autoall/src/autoall/__init__.py` (additions)

- `ensure_all_file(path, *, persist=True)` — scans a single `.py` module file (no import required) for public top-level names (functions, classes, constants) and writes/updates an `__all__ = [...]` block between the `# --- auto-generated __all__ ---` markers inplace (atomically). Exported as `autoall.ensure_all_file`.

`autoall/` (vendored helper package)

`autoall` is developed as a sibling package in `autoall/` and used by `pyrefactor` to write the `__all__` section:

- `ensure_all(pkg_name, ...)` — package mode: parse the user part of `__init__.py`, respect a user `__all__`, otherwise scan the directory and write the block once.
- `ensure_all_file(path, *, persist=True)` — **file mode used by `pyrefactor`**: parse a single module with `ast`, collect public top-level names and write/update the marker-delimited `__all__` block in place (atomic replace, idempotent, CRLF preserving, user `__all__` respected).
- `build_all_block(names)` — public helper returning the exact block text, so `--check`/`--stdout` can show what will be written.
- `install`/`uninstall`/`is_installed`/`regenerate` and the `autoall` CLI.

`arena/` (session helper scripts)

Script	Purpose
setup.sh	create the venv, install the project + test/docs deps
run_tests.sh	run pytest (optionally with coverage)
check_all.sh	tests + docstring audit + docs + PDF + demos
demo_consolidate.sh	run pyrefactor over the example packages

Script	Purpose
test_on_packages.sh	run pyrefactor over real packages (autoall, this repo, examples) and compile the result
verify_prompt.py	assert the prompt.md contract (layout, temp files, __all__, ordering)
helpers.py	small python helpers (make demo packages, validate output)
docstring_audit.py	report missing module/class/function docstrings
add_docstrings.py	libcst script that inserted the missing docstrings
build_docs.sh	build the mkdocs HTML site
build_pdf.py	build refactor_utils.pdf (fpdf2, no browser needed)

Documentation build

`docs/README.md` documents the two-step build (HTML site + single-file PDF), including the tools required and how to verify the result.

Overview

```
cli.py → core.py orchestrates
        → discovery.py (SourceFile, DataFile, pyproject)
        → parser.py (tree-sitter, parallel, picklable ModuleInfo)
        → resolver.py (rename/dedupe/imports/__all__, env/eager)
        → ordering.py (topo sort)
        → writer.py (mode renderers)
        → imports.py (import section)
        → logging_setup.py (loguru)
models.py — picklable dataclasses
```

Flow

1. **CLI** `build_parser()` parses flags, resolves aliases, calls `consolidate()`.
2. **Discovery** `discover()` walks `detect_base(root)` (`src/` or `.`), filtering `EXCLUDED_*` with `include` precedence, skipping `output`, handling single-file/symlink/namespace.
3. **Data** `discover_data_files()` collects non-py data.
4. **Parse** `_parse_all()` parallel (ProcessPool picklable worker `_parse_worker` + BrokenExecutor fallback to Thread, tqdm optional) → `dict[module, ModuleInfo]`.
5. **Resolver** builds `env` (def + import table + star), analyses `eager`/`proj_refs`/`ext_keys`, dedupes (`none|safe|aggressive`), assigns names (`prefix|suffix|module_path|error`, `main_name`), resolves import conflicts, dedupes again with post-rename identities (`_dedupe_post_rename` → `_block_signature`, fixed point), renders via `_render_block` (replacing `Occ`s), assembles `Plan` (topo_order by rank, tail `main`, guards, exports via `all_mode`).
6. **Writer** renders per `mode` (single/package/module/flat/split) with optional `source_map`/`preserve_modules`/`namespace`/`format`, always `compile()` checked.
7. **Core** handles `dry_run`/`check`/`stdout`/`diff`/`overwrite`/`validate`/`copy_data`, copies data via `shutil.copy2`, returns `ConsolidationResult`.

Decoupling

Parser only emits `models.py` types (never tree-sitter nodes); resolver consumes them. Workers never import `loguru` heavy state beyond warnings list.

File Map

See Project Files for per-file docs.

See Also

- `tests/test_integration.py` — end-to-end large/circular/src-layout cases.
- `ordering.topo_order` — cycle handling.

Edge Cases

Full plan from `prompt.md` (see README for table). Each item below is covered by `tests/test_edge_cases.py`.

Dynamic & Metaprogramming

- `importlib`, `__import__`, `globals()`, `locals()`, `eval`, `exec`, `getattr` → `parser._detect_dynamic_patterns` warns; consolidation may miss deps → `Warning`.
- `__getattr__`, `__dir__`, monkeypatching → warns.

Exports & Imports

- Computed/mutated `__all__` → warn, fallback to `ignore`. `=` replaces, `+=`/`.append()`/`.extend()` append, `[]` counts as declared, and non-literal values are reported instead of guessed (`ast.literal_eval`, not a regex).
- `__all__` never reaches the output as a constant block.
- Re-exports via `__all__` and aliased re-exports in `__init__.py` (`from .sub import helper as public_helper`) → `resolver._exports` + `Plan.reexports`, rendered as `helper as public_helper`.
- Metadata dunders (`__version__`, ...) are exported; other private names are not.
- Star imports → local expanded via `_star_names`; external kept `from pkg import *` + warn.
- Conditional/platform/optional (`sys.platform`, `try/except ImportError`, `TYPE_CHECKING`) → parser detects, guards kept verbatim; internal imports found inside a guard are hoisted to module level and removed from the guard text.
- `TYPE_CHECKING` blocks keep non-import statements (type aliases, `Protocol` stubs, `TypeVar`s); names they mention stay imported.
- Unresolvable internal imports (compiled extensions, excluded modules) are kept verbatim rather than dropped.
- `from __future__` features are emitted only when `all` modules agree.
- Nested/side-effect-only imports → `nested_import` occ, internal removed, external kept.
- Aliases/collisions → `resolver._assign_names` + `_resolve_ext_conflicts`.

Names & Scoping

- Duplicate identical bodies → `dedupe safe|aggressive`. A second pass runs after renaming (`resolver._dedupe_post_rename`): it fingerprints each block by what its references resolve to (canonical project block, or the identity of the imported object), so blocks that only looked different because of an import alias (`from .models import User as U` vs `from .models import User`) or a conflict-renamed import are still collapsed, and the imports only those duplicates used are pruned. It runs

to a fixed point and never merges blocks with different names or different bodies.

- Class-body, comprehensions, lambdas, walrus, `match`, `global`/`nonlocal` → `parser._collect_occs` shadow/lazy/cls handling.
- `for` targets, `with ... as`, `except ... as`, `match` captures and comprehension variables shadow module-level names, so a rename never touches them; the class referenced by a `case Point():` pattern is renamed.
- `nonlocal x` is tracked by `parser._function_shadow`: the name is shadowed in the enclosing function too, so the local binding, the nested uses and the declaration itself keep one spelling (a rename can no longer split the variable or point `nonlocal` at a different scope). `global x` removes the inherited shadow, so it follows the renamed module-level object instead of staying stale behind a `for`/`with` binding.
- Using an imported module as a value (`print(utils)`) is a hard error; string forward references to renamed objects produce a warning.
- Decorators/annotations/default args → pushed as `shadow` + `lazy` (future_annotations).

Runtime

- `__file__`/`__name__`/`__package__`/`__path__` + resource loading → warnings (the `__file__` warning points at `--copy-data`).
- `logging.getLogger(__name__)`, `pickle`/`dill`/`shelve`/`marshal` usage → warnings about the collapsed logger hierarchy and `__module__` changes.
- Module identity/order → `topo_order` rank.
- `__main__` guards/multi-entry/`main()` → `MAIN` kind, `--entry*`, `tail` mains, `generate_guard` (which uses `asyncio.run` for `async def main`).

Discovery & IO

- Namespace, `src/` layout, single-file, symlinks (deduplicated by resolved path), output-inside-source → `discovery.py`.
- `__main__.py` is still excluded, but a warning states that `python -m pkg` no longer works in the consolidated output.
- An output path that is a file fails with a `ConsolidationError`; `--check` fails when there is nothing to compare; `--eol` controls line endings.
- Syntax errors/partial parse → `parser` errors vs `allow_syntax_errors` warnings.
- Encoding/BOM/CRLF/tabs/shebang → parser handles.
- `pyproject.toml`/`setup.py` → `detect_pyproject_config`.
- Huge/worker crash/pickling → `core._parse_all` BrokenExecutor

fallback; keep picklable models.

- Permission/non-UTF8 → `cannot read` errors, skipped, `max_skipped`/`strict`.

refactor_utils

refactor_utils: consolidate python packages into a single file.

cli

Command line interface for `pyrefactor`.

Usage::

```
pyrefactor [pkg path]      # path defaults to "."
```

Thin argument-parsing layer over :func:`refactor_utils.core.consolidate_package`; all defaults and behaviour live in :mod:`refactor_utils.core` so the library API and the CLI can never drift apart. `[tool.pyrefactor]` values from `pyproject.toml` are merged in by :func:`apply_config` (explicit command line flags always win).

`apply_config(args: argparse.Namespace, parser: argparse.ArgumentParser) -> argparse.Namespace`
function

Merge `[tool.pyrefactor]` defaults into parsed command line arguments.

`[tool.pyrefactor]` is read from `--config` when given, otherwise from the nearest `pyproject.toml` above the source path. Config values only fill in options the user did not pass explicitly (a value equal to the argparse default is treated as "not passed"), so command line flags always win.

Args: args: Parsed command line namespace (modified in place). parser: The parser, used to look up per-option defaults.

Returns: The same namespace, for convenience.

`build_parser() -> argparse.ArgumentParser`
function

Build the `pyrefactor` argument parser.

Every flag documents the module that implements it, which keeps the CLI help and the library options in sync.

Returns: A fully configured :class:`argparse.ArgumentParser`.

`main(argv: list[str] | None = None) -> int`
function

Run the command line entry point.

Args: argv: Argument list to parse; defaults to `sys.argv[1:]`.

Returns: Process exit code: `0` success, `1` unexpected failure or strictness violation, `2` for a :class:`ConsolidationError`. options: show_root_heading: true

core

Orchestration: discover -> parse (parallel) -> resolve -> write.

Two entry points live here:

- :func:`consolidate_package` implements the `pyrefactor [pkg]` contract: one `consolidated/<pkgname>.py` file built from the temporary `const.py`/`classes.py`/`func.py` extractions, with an `__all__` section added by :mod:`autoall`.
- :func:`consolidate` is the lower-level, multi-layout library API kept for backwards compatibility (`single`/`package`/`module`/`flat`/ `split`).

`ConsolidationError`

class

Raised when a consolidation run cannot produce trustworthy output.

This is the only exception the CLI maps to a dedicated exit code, so all user-facing failures (bad input, unsafe plan, failed `--check`) use it.

`ConsolidationResult(output_dir: pathlib.Path, files: list[pathlib.Path], modules: int, skipped: list[str] = <factory>, exports: int = 0, warnings: list[str] = <factory>, diff: str = '', stdout_text: str = '', copied_data_files: list[pathlib.Path] = <factory>, validated: bool = False, dry_run: bool = False, plan: refactor_utils.resolver.Plan | None = None, all_added: bool = False, pkg_name: str = '') -> None`

class

Summary of a completed (or dry-run) consolidation.

Attributes: output_dir: Directory (or file, in module mode) that was written to. files: Files that were written. modules: Number of modules that contributed to the output. skipped: Paths of files that could not be parsed. exports: Number of names exported by the generated `__init__.py`. warnings: Warnings collected from the parser and the resolver. diff: Unified diff produced by `--diff`/`--check`. stdout_text: Content that was printed instead of written (`--stdout`). copied_data_files: Data files copied next to the output. validated: Whether `--validate` passed. dry_run: Whether files were written. plan: The underlying plan, for `--dry-run` introspection. all_added: Whether :mod:`autoall` appended an `__all__` section. pkg_name: Detected package name (`consolidate_package` only); also the file name of the generated module.

`consolidate(source: pathlib.Path, output: pathlib.Path, *, mode: str = 'single', workers: int | None = None, executor: str = 'process', exclude: tuple[str, ...] = (), include: tuple[str, ...] = (), keep_unused_imports: bool = False, respect_all: bool = False, entry: tuple[str, ...] = (), dry_run: bool = False, check: bool = False, stdout: bool = False, config: pathlib.Path | None = None, do_format: bool = False, formatter: str = 'black', diff: bool = False, overwrite: bool = False, force: bool = False, keep_docstrings: bool = True, strip_comments: bool = False, keep_comments: bool = True, rename_strategy: str = 'prefix', dedupe: str = 'safe', imports_mode: str = 'prune', all_mode: str = 'ignore', namespace: bool = False, preserve_modules: bool = False, source_map: bool = False, fail_on_warning: bool = False, max_skipped: int | None = None, allow_syntax_errors: bool = False, no_parallel: bool = False, progress: bool = False, validate: bool = False, validate_import: bool = False, copy_data: bool = True, data_glob: tuple[str, ...] = (), entry_keep_all: bool = False, main_name: str | None = None, generate_guard: bool = False, strict: bool = False, eol: str = 'lf') -> refactor_utils.core.ConsolidationResult`

function

Consolidate a python module, package or project into fewer files.

Discovery, parsing (optionally parallel), cross-module resolution, rendering and validation all happen here; the CLI is a thin wrapper around this function.

Args: source: File or directory to consolidate. output: Destination directory (or `.py` file in module mode). mode: Output layout: `single`, `package`, `module`, `flat` or `split`. workers: Number of parsing workers; defaults to the CPU count. executor: `"process"` or `"thread"` pool for parsing. exclude: Additional glob patterns to skip. include: Glob patterns that re-include otherwise excluded files. keep_unused_imports: Deprecated alias for `imports_mode="keep"`. respect_all: Deprecated alias for `all_mode="respect"`. entry: Modules whose `__main__` guard should be kept. dry_run: Print the plan without writing files. check: Fail when the output would change (or does not exist yet). stdout: Write the generated source to stdout instead of files. config: Explicit `pyproject.toml` to read `[tool.pyrefactor]` from. do_format: Run black/ruff over the generated files. formatter: Formatter to use with `do_format`. diff: Emit a unified diff against the existing output. overwrite: Allow overwriting a non-empty output directory. force: Alias for `overwrite`. keep_docstrings: Keep docstrings in the output. strip_comments: Drop standalone comment lines. keep_comments: Keep comments at all. rename_strategy: Collision policy: `prefix`, `suffix`, `module_path` or `error`. dedupe: Duplicate removal: `none`, `safe` or `aggressive`. imports_mode: Import handling: `prune`, `keep` or `strict`. all_mode: `__all__` handling: `ignore`, `respect`, `merge` or `recompute`. namespace: Emit module namespace comments. preserve_modules: Emit per-block origin comments. source_map: Emit `file:line` comments per block. fail_on_warning: Fail when any warning was emitted. max_skipped: Fail when more than this many files had to be skipped. allow_syntax_errors: Parse files with syntax errors partially. no_parallel: Force single-process parsing. progress: Show a tqdm progress bar while parsing. validate: Compile (and optionally import) the generated

files. validate_import: With `validate`, also import the generated module. copy_data: Copy non-python data files next to the output. data_glob: Only copy data files matching these globs. entry_keep_all: Keep every `__main__` guard. main_name: Rename `main` to this name. generate_guard: Synthesize a `__main__` guard when none exists. strict: Promote risky warnings and import cycles to hard errors. eol: Line endings for generated files: `lf`, `crlf` or `preserve`.

Returns: A :class:`ConsolidationResult` describing what was (or would be) written.

Raises: ConsolidationError: When the input cannot be parsed, the plan cannot be built safely, or a requested check/validation fails.

consolidate_package(source: pathlib.Path | str = '.', output: pathlib.Path | None = None, *, workers: int | None = None, executor: str = 'process', exclude: tuple[str, ...] = (), include: tuple[str, ...] = (), keep_docstrings: bool = True, strip_comments: bool = False, keep_comments: bool = True, rename_strategy: str = 'prefix', dedupe: str = 'safe', imports_mode: str = 'prune', all_mode: str = 'ignore', entry: tuple[str, ...] = (), entry_keep_all: bool = False, main_name: str | None = None, generate_guard: bool = False, keep_unused_imports: bool = False, respect_all: bool = False, fail_on_warning: bool = False, max_skipped: int | None = None, allow_syntax_errors: bool = False, no_parallel: bool = False, progress: bool = False, dry_run: bool = False, check: bool = False, validate: bool = False, validate_import: bool = False, stdout: bool = False, copy_data: bool = False, data_glob: tuple[str, ...] = (), strict: bool = False, eol: str = 'lf', namespace: bool = False, preserve_modules: bool = False, source_map: bool = False, diff: bool = False, do_format: bool = False, formatter: str = 'black') -> refactor_utils.core.ConsolidationResult

function

Consolidate a package into a single `consolidated/<pkgname>.py` file.

Implements the `pyrefactor <pkg path>` contract:

- the package name is detected from packaging metadata (`pyproject.toml`/`setup.cfg`/`setup.py`) in the package folder and up to two directory levels above it (covering `src/<pkgname>` layouts); when nothing is found, the folder that was pointed at (`.` by default) gives the name. Names are normalised to a valid python identifier, so `my-dist` becomes `my_dist`;
- a `consolidated` subdirectory is created inside the package directory (or at `output` when given) containing a single `<pkgname>.py`; any stale file left by an earlier run is removed;
- constants, functions and classes are extracted from the whole package, name collisions are resolved by the resolver, and each group is first saved to a temporary file (`const.py`, `func.py`, `classes.py`);
- imports are extracted, de-duplicated, local (relative) imports stripped by the resolver, and the optimized import section is written first to `consolidated/<pkgname>.py`;
- the temporary `const.py`/`classes.py`/`func.py` contents are appended and the temporary files are removed;
- :mod:`autoall` then updates the file **in place**, appending a marker-delimited `__all__` section.

Args: source: Package directory to consolidate (a single `.py` file is also accepted); defaults to the current directory. output: Output directory; defaults to `source / "consolidated"`. The single `<pkgname>.py` file is written inside it. workers: Number of parsing workers; defaults to the CPU count. executor: `"process"` or `"thread"` pool for parsing. exclude: Additional glob patterns to skip. include: Glob patterns that re-include otherwise excluded files. keep_docstrings: Keep docstrings in the output. strip_comments: Drop standalone comment lines. keep_comments: Keep comments at all. rename_strategy: Collision policy: `prefix`, `suffix`, `module_path` or `error`. dedupe: Duplicate removal: `none`, `safe` or `aggressive`. imports_mode: Import handling: `prune`, `keep` or `strict`. all_mode: `__all__` handling before autoall runs: `ignore`, `respect`, `merge` or `recompute`. entry: Modules whose `__main__` guard should be kept. entry_keep_all: Keep every `__main__` guard instead of filtering. main_name: Rename the consolidated `main` function to this name. generate_guard: Synthesize a `__main__` guard when none exists. keep_unused_imports: Deprecated alias for `imports_mode="keep"`. respect_all: Deprecated alias for `all_mode="respect"`. fail_on_warning: Fail when any warning was emitted. max_skipped: Fail when more than this many files were skipped. allow_syntax_errors: Parse files with syntax errors partially. no_parallel: Force single-process parsing. progress: Show a tqdm progress bar while parsing. dry_run: Show the plan and stats without writing files. check: Fail when the generated file would differ from the existing one. validate: Compile the generated file. validate_import: With `validate`, also import the generated module. stdout: Write the generated source to stdout instead of files. copy_data: Copy non-Python data files next to the output. data_glob: Only copy data files matching these globs. strict: Promote risky warnings and import

cycles to hard errors. eol: Line endings for generated files: `lf`, `crlf` or `preserve`. namespace: Emit module namespace comments per block. preserve_modules: Emit per-block origin comments. source_map: Emit `file:line` comments per block. diff: Emit a unified diff against the existing output file. do_format: Run black/ruff over the generated file. formatter: `"black"` (default) or `"ruff"`.

Returns: A :class:`ConsolidationResult` describing what was written.

Raises: ConsolidationError: When the input cannot be parsed or the plan cannot be built safely.

discovery

Find the python files of a project and derive their dotted module names.

Also owns package-name detection for the `consolidated/<pkgname>.py` layout (:func:`detect_pkg_name`/:func:`normalize_pkg_name`) and the optional `[tool.pyrefactor]` configuration lookup (:func:`detect_pyproject_config`).

`detect_base(root: pathlib.Path) -> pathlib.Path`
function

Determine the directory that module names are relative to.

Handles both flat package layouts (`pkg/`) and `src/` layouts.

Args: root: Source root passed by the user.

Returns: The directory to compute dotted module names from.

`detect_pkg_name(root: pathlib.Path) -> str`
function

Detect the consolidated package name for `root`.

Investigates `root` and up to two directory levels above it (to cover `src/<pkgname>` layouts) for packaging metadata (`pyproject.toml`, `setup.cfg`, `setup.py`) and returns the declared project/package name. When nothing is found, falls back to `root`'s own folder name (the folder `pyrefactor` was pointed at, i.e. `.` when run inside a package).

The result is always a valid python identifier so that it can be used directly as `consolidated/<pkgname>.py` (`my-dist` becomes `my_dist`).

Args: root: Package directory being consolidated.

Returns: The detected package name, or the folder name as a fallback.

`detect_pyproject_config(root: pathlib.Path) -> dict`
function

Read [tool.pyrefactor] from pyproject.toml if present, for --pyproject awareness.

Returns a dict of config values (exclude, include, entry, etc.) or empty.

`discover(root: pathlib.Path, exclude: tuple[str, ...] = (), skip_dirs: Iterable[pathlib.Path] = (), include: tuple[str, ...] = (), follow_symlinks: bool = False) -> list[refactor_utils.models.SourceFile]`
function

Find the python files of a project and derive their module names.

Symlinks are de-duplicated by resolved path so that a link and its target cannot be parsed as two different modules. Excluded `__main__.py` files produce an explicit warning because `python -m pkg` stops working.

Args: root: File or directory to scan. exclude: Extra glob patterns to skip (highest precedence). skip_dirs: Directories to skip entirely (e.g. the output directory). include: Globs that re-include otherwise excluded paths. follow_symlinks: Recurse into symlinked directories.

Returns: Discovered source files with their dotted module names.

`discover_data_files(root: pathlib.Path, exclude: tuple[str, ...] = (), include: tuple[str, ...] = (), skip_dirs: Iterable[pathlib.Path] = (), copy_data: bool = True, data_glob: tuple[str, ...] = ()) -> list[refactor_utils.models.DataFile]`
function

Discover non-Python data files that should be preserved.

This respects the same exclude/include precedence as :func:`discover` but looks for data extensions and well-known data filenames.

Args: root: Source file or directory that was consolidated. exclude: Extra glob patterns to skip. include: Glob patterns that re-include excluded entries. skip_dirs: Directories never to walk (for example the output directory, which may live inside the source tree). copy_data: When `False` the feature is disabled and an empty list is returned. data_glob: When non-empty, only data files matching one of these globs are returned (matched against the file name and its relative path).

Returns: Data files to copy, each paired with its path relative to the package root.

`normalize_pkg_name(name: str) -> str`
function

Turn a distribution/folder name into a valid python module name.

Packaging names may contain dots and dashes (`my-dist`, `my.pkg`), which cannot be used as a file/module name. Everything that is not a letter, digit or underscore becomes _; a leading digit is prefixed with _; empty results fall back to `"consolidated"`.

Args: name: Raw distribution name (`pyproject.toml [project].name`), folder name, or module stem.

Returns: A valid, importable identifier.

parser

tree-sitter based extraction of top-level blocks (runs inside worker processes).

parse_import(node: tree_sitter.Node, module: str, is_pkg: bool) -> list[refactor_utils.models.ImportSpec]

function

Convert an import statement node into :class:`ImportSpec` objects.

Relative imports are resolved against `module` so downstream code only ever deals with absolute module names.

Args: node: `import_statement`, `import_from_statement` or `future_import_statement` node. module: Dotted name of the module the statement appears in. is_pkg: Whether that module is a package `__init__`.

Returns: One specification per imported name (or a single star specification).

parse_module_file(path_str: str, module: str, is_pkg: bool, keep_docstrings: bool = True, strip_comments: bool = False, keep_comments: bool = True, allow_syntax_errors: bool = False) -> refactor_utils.models.ModuleInfo

function

Parse one python file into a :class:`ModuleInfo`.

Reads the file honouring PEP 263 encoding cookies, re-encodes the text as utf-8 (so byte offsets stay consistent), extracts blocks, imports, guards and `__all__`, and collects warnings. Failures are reported through `ModuleInfo.errors` rather than raised, so a worker never crashes.

Args: path_str: Path to the file to parse. module: Dotted module name the file maps to. is_pkg: Whether the file is a package `__init__.py`. keep_docstrings: Keep docstrings in the extracted block sources. strip_comments: Remove standalone comment lines from blocks. keep_comments: Keep comments at all. allow_syntax_errors: Extract what is possible from a broken file.

Returns: The parsed module, with `errors` populated when parsing failed.

resolver

Cross-module resolution: name collisions, import rewriting, dedupe, ordering.

Plan(future_names: list[str], imports_text: str, guards: list[str], blocks: list[refactor_utils.resolver.RenderedBlock], exports: dict[str, list[str]] = <factory>, renamed: list[tuple[str, str, str]] = <factory>, deduped: list[tuple[str, str]] = <factory>, reexports: dict[str, str] = <factory>, warnings: list[str] = <factory>) -> None

class

Everything needed to render the consolidated output.

Attributes: future_names: `__future__` features to emit (uniform across modules). imports_text: Rendered external import section. guards: Verbatim import guards, in output order. blocks: Rendered blocks in dependency order. exports: Public API split into constants, classes and functions. renamed: `(module, original, final)` rename log. deduped: `(module, name)` log of dropped duplicates. reexports: Exported name to the name inside the consolidated module. warnings: Warnings collected while building the plan.

RenderedBlock(section: str, kind: str, text: str, origin: str = '', module: str = '') -> None

class

A block after rewriting, ready to be emitted.

Attributes: section: Target section: `const`, `class`, `func` or `entry`. kind: Original block kind. text: Rewritten block source. origin: `file:line-range` for source maps. module: Module the block came from.

Resolver(modules: dict[str, refactor_utils.models.ModuleInfo], *, keep_unused_imports: bool = False, respect_all: bool = False, entry: Iterable[str] = (), workers: int = 1, rename_strategy: str = 'prefix', dedupe: str = 'safe', imports_mode: str = 'prune', all_mode: str = 'ignore', namespace: bool = False, preserve_modules: bool = False, source_map: bool = False, entry_keep_all: bool = False, main_name: str | None = None, generate_guard: bool = False, strict: bool = False) -> None

class

Turn parsed modules into a single, dependency-ordered plan.

The resolver owns all cross-module reasoning: environment construction, reference analysis, dedupe, renaming, import pruning and rendering.

writer

Render a :class:`~refactor_utils.resolver.Plan` into final source files.

Holds the layout-specific renderers (single, package, module, flat, split) plus the generated `__init__.py` that re-exports the public API. `render_consolidated_single` and :func:`format_source` back the `consolidated/<pkgname>.py` layout used by the `pyrefactor` CLI.

`format_source(text: str, formatter: str = 'black') -> str`
function

Format generated python source with an optional external formatter.

`black` is used when importable (`formatter="black"`), otherwise the `ruff format` executable is tried; `formatter="ruff"` tries ruff first and falls back to black. When neither tool is available the text is returned unchanged, so callers never have to guard the call.

Args: text: Generated python source. formatter: Preferred formatter: `"black"` (default) or `"ruff"`.

Returns: The formatted source, or `text` when no formatter is available.

`render_consolidated(plan: refactor_utils.resolver.Plan, source_name: str, namespace: bool = False, preserve_modules: bool = False, source_map: bool = False, do_format: bool = False, formatter: str = 'black') -> str`
function

Render the plan as one dependency-ordered module.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source, used in the docstring. namespace: Emit `# namespace:` comments per block. preserve_modules: Emit `# Original module:` comments per block. source_map: Emit `# source: file:line` comments per block. do_format: Run an external formatter over the result. formatter: `"black"` or `"ruff"`.

Returns: The generated module source.

`render_consolidated_single(plan: refactor_utils.resolver.Plan, source_name: str, namespace: bool = False, preserve_modules: bool = False, source_map: bool = False) -> str`
function

Render the plan as one consolidated module file.

Layout (matching the `consolidated/<pkgname>.py` contract):

`__future__` section → optimized (deduped, local-imports stripped) import section → `TYPE_CHECKING` guard → constants → classes → functions → entry points. Sections with no blocks are omitted.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source, used in the docstring. namespace: Emit `# namespace:` comments per block. preserve_modules: Emit `# Original module:` comments per block. source_map: Emit `# source: file:line` comments per block.

Returns: The generated single-module source.

`render_flat(plan: refactor_utils.resolver.Plan, source_name: str, **kwargs: Any) -> str`
function

Render blocks without section headers.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source. **kwargs: `preserve_modules` and `source_map` comment toggles.

Returns: The generated flat module source.

`render_init(plan: refactor_utils.resolver.Plan, source_name: str, module: str = '_consolidated') -> str`
function

Render the package `__init__.py` that re-exports the public API.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source, used in the docstring. module: Module inside the package to import the names from.

Returns: The generated `__init__.py` source.

`render_module(plan: refactor_utils.resolver.Plan, source_name: str, **kwargs: Any) -> str`
function

Render a single module file without a generated `__init__.py`.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source. **kwargs: Forwarded to :func:`render_consolidated`.

Returns: The generated module source.

render_package(plan: refactor_utils.resolver.Plan, source_name: str, **kwargs: Any) -> str
function

Render the consolidated module for package mode.

Args: plan: Plan produced by the resolver. source_name: Name of the consolidated source. **kwargs: Forwarded to :func:`render_consolidated`.

Returns: The generated module source.

render_split(plan: refactor_utils.resolver.Plan, source_name: str) -> dict[str, str]
function

Split mode: return mapping filename -> content for const/class/func/entry splits.

models

Plain, picklable data structures shared by the parser and the resolver.

Block(module: str, kind: str, names: list[str], source: bytes, occs: list[refactor_utils.models.Occ], order: int, bid: int = -1, orig_path: str = '', start_line: int = 0, end_line: int = 0, stripped_docstring: bool = False, name_kinds: dict[str, str] = <factory>, binds_value: bool = True) -> None

class

A top-level, relocatable chunk of source.

Attributes: module: Module the block came from. kind: One of `const`, `class`, `func`, `misc` or `main`. names: Names bound by the block (the primary name comes first). source: Raw block source bytes. occs: Occurrences that may need rewriting. order: Position within the originating module. bid: Stable block id assigned by the resolver. orig_path: Source file path, used by source maps. start_line: First source line of the block. end_line: Last source line of the block. stripped_docstring: Whether docstrings were stripped from the source. name_kinds: Optional per-name kind override (`const`/`class`/`func`) for compound blocks that bind names of multiple kinds. binds_value: Whether the block actually binds runtime values for its `names` (`False` for bare annotations like `X: int`).

ConsolidationOptions(keep_docstrings: bool = True, strip_comments: bool = False, keep_comments: bool = True, allow_syntax_errors: bool = False, rename_strategy: str = 'prefix', dedupe: str = 'safe', imports_mode: str = 'prune', all_mode: str = 'ignore', namespace: bool = False, preserve_modules: bool = False, source_map: bool = False, entry_keep_all: bool = False, main_name: str | None = None, generate_guard: bool = False, strict: bool = False, eol: str = 'lf') -> None

class

Picklable bundle of user options that affect parser/resolver/writer.

Kept in models.py so workers can receive it if needed; but currently most flags are handled in the main process (resolver/writer).

Attributes: keep_docstrings: Whether docstrings survive into the output. strip_comments: Whether standalone comment lines are dropped. keep_comments: Whether comments are kept at all. allow_syntax_errors: Parse files with syntax errors partially. rename_strategy: Collision policy (`prefix`/`suffix`/ `module_path`/`error`). dedupe: Duplicate removal mode (`none`/`safe`/`aggressive`). imports_mode: Import handling (`prune`/`keep`/`strict`). all_mode: `__all__` handling (`ignore`/`respect`/`merge`/ `recompute`). namespace: Emit module namespace comments. preserve_modules: Emit origin comments per block. source_map: Emit `file:line` comments per block. entry_keep_all: Keep every `__main__` guard. main_name: Name to use for the `main` entry point. generate_guard: Synthesize a `__main__` guard when missing. strict: Promote risky warnings to errors. eol: Line ending policy (`lf`/`crlf`/`preserve`).

DataFile(path: 'object', relative: str, kind: str = 'data') -> None

class

Non-Python data file discovered alongside the source tree.

Attributes: path: Filesystem path of the data file. relative: Path relative to the source base, used for output placement. kind: Classification of the file (`"data"` by default).

GuardBlock(text: str, specs: list[refactor_utils.models.ImportSpec], internal: bool = False, spec_lines: list[tuple[refactor_utils.models.ImportSpec, str]] = <factory>) -> None

class

A verbatim `try/except ImportError` (or similar) import guard.

Attributes: text: The guard exactly as it appeared in the source. specs: Import specifications found inside the guard. internal: Whether any of `specs` point at a project module. Internal imports cannot survive verbatim in a consolidated file, so the resolver hoists them out of the guard. spec_lines: `(spec, verbatim statement text)` pairs, used to remove exactly the hoisted statements from `text`.

ImportSpec(module: str, name: str | None = None, alias: str | None = None, is_from: bool = False, star: bool = False, relative: bool = False, type_checking: bool = False) -> None

class

A single import as written in the source.

Attributes: module: Dotted module the name is imported from (resolved to an absolute name for relative imports). name: Imported symbol, or `None` for `import module`. alias: `as` alias, if any. is_from: Whether the statement was a `from ... import ...`. star: Whether the statement was a star import. relative: Whether the statement used a relative (`.`) module. type_checking: Whether the import appeared inside `if TYPE_CHECKING:`.

ModuleInfo(module: str, path: str, is_pkg: bool, imports: list[refactor_utils.models.ImportSpec] = <factory>, blocks: list[refactor_utils.models.Block] = <factory>, tc_blocks: list[refactor_utils.models.Block] = <factory>, guards: list[refactor_utils.models.GuardBlock] = <factory>, declared_all: list[str] | None = None, deleted_names: set[str] = <factory>,

future_annotations: bool = False, future_names: list[str] = <factory>, encoding: str = 'utf-8', warnings: list[str] = <factory>, errors: list[str] = <factory>) -> None

class

Everything the parser extracted from a single source file.

Attributes: module: Dotted module name the file maps to. path: Filesystem path the module was read from. is_pkg: Whether the file is a package `__init__.py`. imports: Module-level import specifications. blocks: Top-level, renameable/orderable source blocks. tc_blocks: Non-import declarations extracted from `if TYPE_CHECKING:` suites, to be merged into a single `if TYPE_CHECKING:` guard. guards: Import guards kept verbatim (`try/except ImportError`). declared_all: Literal contents of `__all__`, or `None` when the module does not declare one. An explicit empty list is preserved. future_annotations: Whether the module uses `from __future__ import annotations`. future_names: All `__future__` features requested by the module. encoding: Encoding detected from the source (PEP 263), default utf-8. warnings: Human readable warnings collected during parsing. errors: Fatal problems; modules with errors are skipped.

Occ(start: int, end: int, name: str, kind: str, eager: bool = True, obj: str = '', specs: tuple[refactor_utils.models.ImportSpec, ...] = ()) -> None

class

A renameable occurrence inside a block.

Offsets are relative to the start of the block source, which makes blocks relocatable and safe to merge.

Attributes: start: Start byte offset inside the block. end: End byte offset inside the block. name: Referenced name. kind: `"name"`, `"attr"` or `"nested_import"`. eager: Whether the reference is evaluated at import time. obj: For `"attr"` occurrences, the dotted object expression. specs: For `"nested_import"` occurrences, the parsed imports.

ResolverError

class

Raised when a consolidation plan cannot be built without breaking code.

Lives here (rather than in `resolver`) so that `ordering` can raise it without importing the resolver module.

SourceFile(path: 'object', module: str, is_pkg: bool) -> None

class

A discovered python source file.

Attributes: path: Filesystem path of the file. module: Dotted module name the file maps to. is_pkg: Whether the file is a package `__init__.py`.

dotted_sort_key(dotted: str) -> tuple[tuple[str, str], ...]

function

Build a segment-wise sort key for dotted module names.

Args: dotted: Dotted module path (for example `"pkg._internal"`).

Returns: Tuple of :func:`ident_sort_key` keys, one per segment.

ident_sort_key(name: str) -> tuple[str, str]

function

Build a deterministic sort key where _ precedes letters and digits.

ASCII places `A–Z` (65–90) and `0–9` (48–57) before _ (95), so plain string sorting puts `Alpha` before `_Alpha` and `_1` before `__a`. Mapping _ to `\x00` orders leading and embedded underscores before alphanumeric characters while keeping comparisons case-insensitive with a deterministic case tie-breaker.

Args: name: Identifier or filename to key.

Returns: A `(casefolded, exact)` tuple suitable for `sorted(..., key=...)`.

Contributing

Workflow

```
git clone ...; pip install -e ".[test,docs]"
pyrefactor src/refactor_utils -o /tmp/out --validate --strict
pytest -q
mkdocs serve
```

Adding a Flag

1. `cli.py build_parser()` — add `add_argument` with help mentioning module.
2. `core.consolidate()` — add param, merge `pyproject.toml` if needed, pass to resolver/parser/writer.
3. `discovery`/`parser`/`resolver`/`writer` — implement_effect.
4. `models.py` — if worker data needed, add field (keep picklable).
5. README + `docs/cli-reference.md` flag table + `docs/edge-cases.md` if needed.
6. `tests/test_cli.py` + `tests/test_core.py` + `examples/`.

Testing

```
pytest
pytest --cov=refactor_utils
pytest -m "not slow"
```

Docs

```
mkdocs serve      # http://127.0.0.1:8000
mkdocs build      # site/
```

Style: Google docstrings (mkdocstrings), Material theme.

Changelog

1.5.0 — `consolidated/<pkgname>.py` layout, docs build, tests

CLI contract (prompt.md §1):

- `pyrefactor [pkg path]` — the path is optional and defaults to `.`. The package name is detected from `pyproject.toml`/`setup.cfg`/`setup.py` in the package folder and up to two levels above (`src/<pkgname>` layouts), falling back to the folder name. Names are normalised to importable identifiers (`normalize_pkg_name`: `my-dist` → `my_dist`).
- One `consolidated/<pkgname>.py` file is produced (the old `output/` + `__init__.py` + `_consolidated.py` layout is no longer written by the CLI). Stale files from a previous run/legacy layout are removed so the directory always holds exactly one file.

Extraction (prompt.md §2):

- `_write_section_files()` saves constants → `const.py`, classes → `classes.py`, functions → `func.py` in the output directory; the optimized, de-duplicated, local-import-stripped import section is written first to `consolidated/<pkgname>.py`, then `_merge_body()` appends the section files (reading them back) and deletes the temporaries.
- `autoall.ensure_all_file()` then updates the file **in place** with the marker-delimited `__all__` section; `autoall.build_all_block()` is the new public helper used by `--check`/`--stdout` so the compared bytes match a real run. `ensure_all_file` is now byte-for-byte idempotent, preserves CRLF, and respects a user-defined `__all__`.

Behaviour and options:

- Every previously parsed but ignored CLI flag now reaches the pipeline: `--entry`, `--entry-keep-all`, `--main-name`, `--generate-guard`, `--keep-unused-imports`, `--respect-all`, `--data-glob`, `--diff`, `--format`/`--formatter`, `--no-parallel`.
- `[tool.pyrefactor]` is applied by `cli.apply_config()` (auto-detected or via `--config`); explicit command-line flags win.
- `--diff` and `--check` compare the exact generated bytes (LF/CRLF aware); `--format` output is reproducible, so `--check --format` passes on a rerun.
- `--copy-data` copies data files package-relative, so `Path(__file__).parent / "data/…"` keeps working from `consolidated/`.
- `data_glob` filtering was added to `discovery.discover_data_files()`.
- Unresolvable relative imports (single-file inputs, no package anchor) are dropped with a warning instead of emitting invalid `from import x` syntax; resolvable relative imports keep the absolute form.
- Validation now compiles the final (post-`__all__`) file and honours `--eol` on read, so `--check` is stable.

Docs (prompt.md §4–5):

- Google-style docstrings added to every remaining module, class, function and method (68 were missing); `arena/docstring_audit.py` verifies coverage.
- `mkdocs build --strict` is clean (`docs/README.md` excluded from the site).
- Single-file PDF (`refactor_utils.pdf`) is built from the mkdocs nav with `arena/build_pdf.py` (fpdf2, no browser/LaTeX), including expanded API reference pages and a table of contents with page numbers.
- `docs/README.md` documents the doc build step by step.

Tests (prompt.md §3):

- `tests/test_consolidated.py` — layout guarantees, section ordering, temporary-file lifecycle, `__all__`, `--check`/`--diff`/`--format`/`--eol`, entry options and data copying.
- `tests/test_cli_workflow.py` — real `main()` runs: no-arg invocation, explicit path, `-o`, config merge, exit codes, stdout/dry-run.
- `tests/test_autoall.py` — block builder, in-place idempotency, CRLF, user `__all__` precedence.

Unreleased — prompt.md: `consolidated/<pkgname>.py` layout

- CLI: `pyrefactor [pkg]` — `source` is optional (defaults to `.`); `--mode` is removed. A single `consolidated/<pkgname>.py` is written (was `output/` with `__init__.py` + `_consolidated.py`).
- Package name detection: `discovery.detect_pkg_name()` reads `pyproject.toml`/`setup.cfg`/`setup.py` in the package folder and up to two directory levels above it (`src/<pkgname>` layouts); falls back to the folder's name.
- Extraction per prompt.md section 2: constants → `const.py`, functions → `func.py`, classes → `classes.py` (temporary), imports are deduplicated with local/relative imports stripped, the groups are assembled into `consolidated/<pkgname>.py` and the temporary files are removed.
- `autoall.ensure_all_file()` appends an `__all__` section to a single `.py` file inplace; used after consolidation to annotate the generated file's API.
- `core.consolidate_package()` implements the new flow; `writer.render_consolidated_single()` renders the single module layout. `core.consolidate()` and the legacy modes remain available as the library API.
- Tests: `tests/test_consolidated.py` covers the new flow; CLI tests updated.

Unreleased — prompt.md 3.4 (`nonlocal`) + 5.2 (post-rename dedupe)

- `global`/`nonlocal` declarations are tracked per scope (`parser._scope_declarations`, `_nested_nonlocals`, `_function_shadow`): a name declared `nonlocal` keeps the same spelling as the enclosing function binding it refers to, instead of

being renamed on one side and left alone on the other (which silently split one variable into two). `global` declarations remove the inherited shadow, so a renamed module-level object is written consistently inside the function (previously `global x` could stay `x` while the module object became `b_x`).

- A second dedupe pass runs after `_assign_names`/`_resolve_ext_conflicts` (`resolver._dedupe_post_rename`, keyed on `_block_signature`). It compares resolve-to-the-same-thing identities instead of pre-rename text, so blocks that differ only by `from .models import User as U` vs `from .models import User`, or by an import alias, are recognised as duplicates. It runs to a fixed point and the imports that only the dropped duplicate used are pruned.

Unreleased — prompt.md follow-up: correctness fixes + docstrings

Correctness (silently wrong output before):

- `from __future__` imports are only emitted when `every` module requests them; partial usage is reported per module instead of applied to all of them.
- `__init__.py` re-exports such as `from .sub import helper as public_helper` are exported (and aliased) by the generated `__init__.py`.
- `__all__` semantics fixed: `=` replaces, `+=`/`.append()`/`.extend()` append, an explicit `[]` counts as declared, entries are parsed with `ast.literal_eval` (unicode names work, dynamic expressions are reported) and `__all__` is never emitted as a constant block.
- Metadata dunders (`__version__`, `__author__`, `__email__`, `__license__`, `__copyright__`) and anything listed in `__all__` are exported.
- Non-import statements inside `if TYPE_CHECKING:` are preserved instead of dropped; internal imports there are hoisted out of the guard.
- Internal imports inside `try/except ImportError` guards are hoisted to module level (with a warning) instead of producing broken output; imports that cannot be resolved (compiled extensions) are kept verbatim instead of dropped.
- `--generate-guard` emits `asyncio.run(main())` for `async def main`.
- Local bindings (`for`, `with`, `except ... as`, `match` captures, comprehension and walrus targets) are no longer rewritten by module-level renames; class references inside `match` patterns still are.
- Non-UTF8 sources honour PEP 263 cookies, are re-encoded as utf-8, and the stale coding cookie is rewritten.
- Duplicate constant definitions are deduplicated in `safe` mode; imports inside function bodies stay local so optional dependencies stay optional.

Safety / IO:

- Using a project module as a value (e.g. `print(utils)`) is a hard error instead of a silent `NameError`; string forward references to renamed

objects warn.

- `--strict` promotes unresolved-name warnings to errors and rejects import cycles; `--check` fails when the output is missing.
- A file at the output path fails with a clear message; symlinks are deduplicated by resolved path; `__main__.py` exclusion is announced; `--eol` was added; `--validate-import` warns that it executes user code.
- New warnings for `pickle`/`logging.getLogger(__name__)`/`__file__`-relative resources, star-import shadowing, and overlapping rewrites (logged at DEBUG).

Docs/tests: Google-style docstrings for every module, class and function across `src/`, `tests/`, `tools/` and `examples/` (`ruff check --select D` is clean), plus `tests/test_edge_cases.py` covering each fix.

1.4.8 — Prompt.md Full + Tests/Examples/Docs

- Implements all `prompt.md` flags (dry-run, stdout, config, format, diff, overwrite, include, docstrings/comments, rename-strategy, dedupe, imports, all, entry, namespace, source-map, fail-on-warning, max-skipped, allow-syntax-errors, jobs, validate, modes, pyproject awareness).
- Edge-case hardening (dynamic, **all**, star, conditional, circular, scoping, runtime, discovery/IO).
- Data-file preservation (`discover_data_files` → copy).
- Added `tests/` (pytest 50+ cases), `examples/` (basic/advanced/programmatic/data/config), `docs/` (mkdocs Material + mkdocstrings), documented every project file.

1.4.7

- Initial AI-generated `pyrefactor` (`single` mode, tree-sitter, parallel, topo ordering, collision rename prefix).

1.4.8-pre

- Added `pyproject.toml [tool.pyrefactor]` example, `src/refactor_utils/data/` examples.